IMAGE PROCESSING & COMPUTER VISION

Final Project Report

Food Calorie Estimator

from Image

Deep Learning · PyTorch · Streamlit Deployment

Team Member	Student ID
Mariam Khalil Ibrahim	230101792
Mahmoud Usama Abdelhameed	230103270
Omar Abdulaal Reda	230103273

GitHub: github.com/Mariam-Nassar/Food-Calorie-Estimator

May 2026

1. Project Overview

This project presents an end-to-end deep learning pipeline for estimating the caloric content of food dishes directly from images. Calorie estimation is framed as a continuous regression problem given an overhead photograph of a food dish; the system outputs a single predicted calorie value (kcal).

Two architectures were built and compared: a custom CNN designed from scratch, and a MobileNetV2-based transfer learning model with a hand-coded regression head. Both models were evaluated on a held-out test set using regression metrics (MAE, MSE, RMSE, R^2). A Streamlit web application was developed and deployed as the bonus component.

1.1 Team Responsibilities

Member	ID	Responsibilities
Mariam Khalil Ibrahim	230101792	Dataset download · metadata linking · image cleaning · resize/normalize · train-val-test split
Omar Abdulaal Reda	230103273	CNN from scratch · MobileNetV2 transfer learning · model training & checkpoint saving
Mahmoud Usama Abdelhameed	230103270	Evaluation metrics · loss curves · comparison table · Streamlit app · slides & report

2. Dataset

2.1 Source

The dataset is Nutrition5k, a large-scale real-world food dataset published by Google Research. It provides overhead RGB images of food dishes with detailed nutritional metadata (calories, macronutrients, ingredient-level annotations). Dataset URL:

https://github.com/google-research-datasets/Nutrition5k

2.2 Subset & Splits

Two files from the Nutrition5k release were downloaded, yielding 1,021 usable images after cleaning. Each image is linked to its calorie label via the dish ID in the metadata CSV. The split below was generated in notebook 01_data_preparation. ipynb:

Split	Images	Percentage
Training	712	69.7 %
Validation	153	15.0 %

Test	153	15.0 %
Total	**1,021**	**100 %**

Target variable: Calories (continuous, kcal) | **Input resolution:** 224 × 224 pixels, RGB

3. Preprocessing

All preprocessing steps were implemented in notebook 01_data_preparation. ipynb and applied before training. No third-party preprocessed datasets were used — all transformations were written from scratch on raw Nutrition5k files.

3.1 Metadata Linking

Each image filename was matched to its calorie entry in the Nutrition5k metadata CSV using the dish ID as the join key. Images with no matching entry or a null calorie label were excluded from the dataset.

3.2 Image Cleaning

Every file was opened with PIL. Files that raised a decoding exception (corrupted or truncated images) were dropped. The remaining images were verified to have three colour channels (RGB).

3.3 Resizing

All images were resized to 224 × 224 pixels — the standard input resolution for both the custom CNN and the MobileNetV2 backbone.

3.4 Normalization

Pixel values were normalized using the ImageNet channel statistics: mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]. This was applied identically to both models for a fair comparison.

3.5 Data Augmentation (training set only)

The following transforms were applied stochastically during training via torchvision.transforms:

- Random horizontal flip
- Random rotation ± 15 degrees
- Color jitter (brightness and contrast variation)

Validation and test sets received only resize + normalize — no augmentation.

3.6 Train / Validation / Test Split

The cleaned 1,021-image corpus was split 70 / 15 / 15 with a fixed random seed (SEED = 42) to ensure reproducibility. Split indices were saved as CSV files for use by both the training notebook and the Streamlit app.

4. Model Architectures

4.1 CNN from Scratch

A four-block convolutional network was designed and coded layer-by-layer in PyTorch (class ScratchCNN). No pre-built blocks or high-level wrappers were used. The architecture is detailed below:

Block	Layers	Output Shape
Block 1	Conv2d(3→32, 3×3, pad=1) + BatchNorm2d + ReLU + MaxPool2d(2)	32 × 112 × 112
Block 2	Conv2d(32→64, 3×3, pad=1) + BatchNorm2d + ReLU + MaxPool2d(2)	64 × 56 × 56
Block 3	Conv2d(64→128, 3×3, pad=1) + BatchNorm2d + ReLU + MaxPool2d(2)	128 × 28 × 28
Block 4	Conv2d(128→256, 3×3, pad=1) + BatchNorm2d + ReLU + MaxPool2d(2)	256 × 14 × 14
FC Head	Flatten → Linear(50176→512) + ReLU + Dropout(0.5) → Linear(512→128) + ReLU + Dropout(0.25) → Linear(128→1)	1 (kcal)

Loss: MSE | **Optimizer:** Adam | **Epochs:** 10 | **Batch size:** 16 | **Parameters:** 26,145,793

4.2 Transfer Learning — MobileNetV2

The MobileNetV2 backbone was loaded with ImageNet pretrained weights. The original classifier head was removed. A custom regression head was added explicitly in code — not via a single drag-and-drop call. Training proceeded in two phases:

Phase 1 — Frozen backbone (5 epochs)

All backbone parameters were frozen (requires_grad = False). Only the custom head was trained, allowing it to align to calorie regression without disturbing pretrained features.

Phase 2 — Fine-tuning (5 epochs)

The last layers of the backbone were unfrozen and trained jointly with the head at a reduced learning rate, allowing domain-specific adaptation of higher-level feature representations.

Custom regression head (every layer explicitly defined in code):

Layer	Configuration
Dropout	p = 0.3
Linear	in_features → 256
ReLU	—
Dropout	p = 0.2
Linear (output)	256 → 1 (calorie prediction)

Total epochs: 10 (5 frozen + 5 fine-tuning) | **Trainable parameters:** 1,854,273

5. Evaluation

Both models were evaluated on the 153-image test set that was never seen during training or validation. All metrics were computed on this set and saved to results/model_comparison.csv.

5.1 Metrics

Metric	Formula	Interpretation
MAE	Mean $\|y - \hat{y}\|$	Average absolute calorie error (kcal) directly interpretable
MSE	Mean $(y - \hat{y})^2$	Penalises large errors heavily; used as training loss
RMSE	√MSE	Same unit as calories (kcal); easier to compare with MAE
R^2	1 − SS_res/SS_tot	Variance explained: 1.0 = perfect, 0 = baseline mean predictor

5.2 Test-Set Results

Model	MAE (kcal)	MSE	RMSE (kcal)	R^2
CNN from scratch	**104.91**	**20,293.33**	**142.45**	**0.604**
MobileNetV2 TL	149.85	39,825.13	199.56	0.223

Bold green values indicate the better result per metric. The CNN from scratch outperforms MobileNetV2 on every metric.

5.3 Training History — CNN from Scratch (10 epochs)

Epoch	Train Loss (MSE)	Val Loss (MSE)
1	45,152.08	34,370.25
2	31,728.79	17,642.25
3	26,039.05	16,117.28
4	28,865.29	20,069.00
5	24,852.80	21,669.46
6	25,491.17	13,734.60
7	24,179.03	17,434.38
8	20,750.85	14,726.38
9	21,580.40	12,207.13
10	19,036.80	15,681.00

5.4 Training History — MobileNetV2(10 epochs)

Epoch	Phase	Train Loss (MSE)	Val Loss (MSE)
1	Frozen backbone	113,194.95	70,881.58
2	Frozen backbone	91,291.00	49,443.56
3	Frozen backbone	64,713.62	32,396.80
4	Frozen backbone	51,165.51	29,348.48
5	Frozen backbone	48,170.80	28,692.68
1	Fine-tuning	46,487.92	27,048.48
2	Fine-tuning	42,740.62	25,368.69
3	Fine-tuning	40,376.93	24,321.00
4	Fine-tuning	38,872.33	22,995.39
5	Fine-tuning	35,723.63	22,117.90

6. Model Comparison

Criterion	CNN Scratch	MobileNetV2	Unit	Better Model
Trainable parameters	26,145,793	1,854,273	count	**MobileNetV2 (14× fewer)**
Training time	166.24 s	125.46 s	seconds	**MobileNetV2**
MAE	**104.91**	149.85	kcal	**CNN Scratch ✓**
MSE	**20,293.33**	39,825.13	kcal²	**CNN Scratch ✓**

RMSE	**142.45**	199.56	kcal	**CNN Scratch ✓**
R^2	**0.604**	0.223	—	**CNN Scratch ✓**

CNN from scratch achieved stronger regression performance on all four-accuracy metrics. Despite having 14 times more parameters, it converged to a significantly lower error rate (R^2 = 0.604 vs. 0.223). MobileNetV2 was faster to train and lighter in weight — advantages in resource-constrained deployment — but its classification-oriented ImageNet features did not transfer as effectively to the continuous calorie regression task within the given training budget. Additional fine-tuning epochs or unfreezing a larger portion of the backbone would likely close this gap.

7. BONUS Streamlit Deployment [Completed]

> **★ Bonus Component — Full Marks Claimed ★ This section documents the complete, functional web-application deployment of the project.**

A fully functional web application was built using Streamlit (app.py — 366 lines of documented code). The app loads both trained model checkpoints at startup and exposes a multi-tab interface covering all project stages. It is launched via streamlit run app.py or the included run_app.ps1 PowerShell script.

7.1 Application Tabs

Tab	Contents
Overview	Project description, architecture summary, GitHub link
Metrics & Curves	Side-by-side comparison table + bar charts of MAE, RMSE, R^2 for both models
Dataset	Train / val / test split counts table + visual image preview grid per split
Predict Calories	Upload any food image → select CNN or MobileNetV2 → click Estimate → get predicted kcal. For known dataset images, the ground-truth label is also shown alongside the prediction.

7.2 Technical Stack

Component	Technology
Web framework	Streamlit
Deep learning	PyTorch + torchvision
Image handling	Pillow (PIL)
Data & analysis	pandas, numpy
Models served	cnn_from_scratch.pth + MobileNetV2 weights
Launch script	run_app.ps1 (Windows PowerShell)

7.3 Evidence of Deployment

The following screenshots from the running application confirm a complete, functional deployment:

- Dataset tab showing the split counts (712 / 153 / 153) with food-image preview grid
- Predict Calories tab showing a real dish image (bread + banana), the model dropdown, and the Estimate Calories button with Actual Calories = 271.32 kcal displayed

Both screenshots were captured from the live running Streamlit app and are included in the project submission as evidence.

8. Conclusion & Recommendations

8.1 Key Findings

- The CNN from scratch delivered the best regression results: $R^2 = 0.604$, MAE = 104.91 kcal, RMSE = 142.45 kcal on the unseen test set.
- MobileNetV2 transfer learning converged faster (125 s vs. 166 s) and used far fewer parameters (1.85 M vs. 26.1 M), but showed weaker generalisation to this regression task ($R^2 = 0.223$).
- The Streamlit deployment is fully functional and supports real-time calorie estimation with either model (Bonus — completed in full).

8.2 Limitations

- Dataset size (1,021 images) is relatively small for a continuous regression task. Calorie variance is high and larger data would improve generalisation.
- 2D overhead images cannot capture portion depth or weight — key factors in actual calorie content.
- MobileNetV2 was not fine-tuned for enough epochs; more cycles or a larger unfrozen block would likely close the performance gap.

8.3 Recommendations

- Download additional Nutrition5k files to expand the training set toward the full 5,000-image corpus.
- Experiment with EfficientNet-B0 or ResNet50 as alternative backbones for transfer learning.
- Explore multi-task learning — predicting protein, fat, and carbohydrates simultaneously alongside calories.
- Add auxiliary inputs (estimated weight, depth map) to improve calorie prediction accuracy.
- Deploy the Streamlit app publicly via Streamlit Community Cloud.

9. Repository Structure

Path	Contents
/data/	Raw images and processed split CSV files
/notebooks/	01_data_preparation.ipynb · 02_model_training.ipynb
/models/	Saved weights: cnn_from_scratch.pth, cnn_from_scratch_full.pt, MobileNetV2 files
/results/	model_comparison.csv · cnn_training_history.csv · mobilenet_v2_training_history.csv
app.py	Streamlit deployment application (366 lines)
train_models.py	Full training pipeline script (265 lines)
download_images.py	Nutrition5k image downloader
requirements.txt	All Python dependencies
README.md	Setup and run instructions